

Nicholas Tierney

R Best Practices

r-best-practices.njtierney.com

Table of contents

Welcome 5

Course materials 7

Schedule 9

Project Organisation 9

Code style and readability 9

Writing readable code 9

Writing Functions 9

Writing a reprex / getting unstuck 10

Putting It All Together 10

Setup 11

How this book works 11

The book is longer than the course 11

How we will work 11

The boxes 11

The code 13

Where everything lives 13

What to install 14

R 14

The R packages 15

Code formatters 15

Jarl, the linter 16

Checking it all worked 16

If something will not install 17

1 Project organisation 19

1.1 Overview 19

1.2 Questions 19

1.3	<i>A common problem: file paths</i>	19
1.3.1	<i>Files</i>	20
1.3.2	<i>Directories</i>	20
1.3.3	<i>Paths</i>	20
1.4	<i>tree</i>	21
1.5	<i>file explorer</i>	21
1.6	<i>What does <code>setwd()</code> do?</i>	23
1.7	<i>Is there an answer to the madness?</i>	24
1.8	<i>RStudio</i>	24
1.9	<i>Positron</i>	26
1.10	<i>The {here} package</i>	26
1.11	<i>Why organise your project?</i>	27
1.12	<i>Pitfalls of organisation</i>	28
1.12.1	<i>Everything in one folder</i>	29
1.13	<i>Unordered</i>	29
1.14	<i>Ordered</i>	30
1.14.1	<i>No clear entry point</i>	30
1.15	<i>File names encode history, not content</i>	31
1.16	<i>Editing raw data in place</i>	31
1.17	<i>Outputs mixed in with inputs</i>	32
1.18	<i>Common project structures</i>	32
1.18.1	<i>Naming the files themselves</i>	34
1.19	<i>“Good enough” common sense principles of project organisation</i>	35
1.20	<i>Always have a README</i>	35
1.21	<i>RStudio and Positron set up</i>	37
1.22	<i>RStudio</i>	37
1.23	<i>Positron</i>	38
1.24	<i>Summary</i>	39

Welcome

Welcome to R Best Practices! This course covers essential best practices for writing clear, maintainable, and reproducible R code.

Course materials

Materials will be added here as we progress through the course.

<https://r-best-practices.njtierney.com>

Prerequisites

- Basic R programming experience
- Familiarity with writing R scripts
- Experience working on data analysis projects

Learning outcomes

- How to name things effectively
- Using a style guide
- How to refactor your code
- How to review your code and others'
- How to lay out a project so others know how to run your code
- How to make a reproducible example (reprex)

Schedule

Project Organisation

- Understanding file paths
- Pitfalls of organisation
- Common project structures
- Naming files
- “Good Enough”/Common Sense principles of project organisation
- Always have a README
- RStudio and positron set up

Code style and readability

- Style is like grammar
- Layout, naming, and correctness are three different jobs
- Why we care about consistent names, spaces, and indentation
- Using the {air} formatter
- Using linters
- Bonus: typing skills and keyboard shortcuts

Writing readable code

- Where we are up to: what the tools fixed, and what they didn't
- Good vs bad variable names
- De-chunking code
- What to look for: code smells, inputs and outputs, complexity, refactoring
- A process for reviewing code
- Reviewing someone else's code
- Clean as you go

Writing Functions

- The problem functions solve
- Anatomy of a function
- Good (simple) function design
- Using {fnnmate} to speed up creating functions
- How to use a debugger

Writing a reprex / getting unstuck

- How to share problems
- Using {reprex}
- Practicing reprex
- Practical tips on debugging
 - Keep solving vs cleaning up

Putting It All Together

- Take an existing project and provide code review
- Apply code linting, code styling, functions
- Discussion and Q & A

Setup

Two things before we start:

1. How this book is put together, and
2. What to install.

The first part takes two minutes to read and will make the rest of the course easier to follow.

How this book works

The book is longer than the course

The course is six hours. The book has more in it than six hours can cover, and that is on purpose.

We follow one path through it live, and the rest stays here for when you want it. So if we skip a section, or if you notice a box we never opened, nothing has gone wrong - it is there for you to come back to. I would rather write the complete version and choose what to teach on the day than write only what fits and leave you with notes that stop where the session did.

How we will work

Mostly by typing. I will write code, you write it too, and we will break things together and then fix them. You will get more out of this by typing the examples than by watching me type them, even when it feels slower.

You do not need to keep up with every keystroke. Everything on screen is in the book, with a copy button on every code block.

The boxes

Four kinds of box turn up throughout, each with its own colour and icon:

- **Your Turn** - something for you to do. Most of these we do together in the session, and a few are there for afterwards. The ones that ask what you think have no wrong answers; I am genuinely curious what has gone wrong for you before.

- **Some history** - where something came from. Why a linter is called a linter, where the backslash in `\(x)` comes from. Entirely skippable, and quietly my favourite part of the book.
- **Going deeper** - optional extra detail, a longer example, or a list you might want later. Boxes titled **Read more** point at the blog post or talk a section is adapted from, so you can follow the original if you want it.
- - the things that will actually bite you. There are not many. That is the point of them.

Rather than describe them, here is one of each.

Your Turn

Exercises look like this. Most of them give you something to run:

```
R.version.string
```

Some of them just ask you something. Here is the one I actually care about: what is the worst R code you have ever had to work with? Your own counts, and mine definitely does.

There is no wrong answer. I am asking because the answer usually tells me which parts of the day to slow down on.

Some history: where the name R comes from

Two things at once, which I think is lovely.

R was written by Ross Ihaka and Robert Gentleman at the University of Auckland in the early 1990s, and they share a first initial. It is also a play on S, the language it was modelled on, in the same way that C came after B.

So the name is a pun and a signature at the same time. You can read their own account of it in [R: A Language for Data Analysis and Graphics](#).

Going deeper: how these boxes are made

You just clicked a title, so that is the collapsing demonstrated. These are Quarto callouts, and there is nothing clever going on. Each one is a fenced div with a class on it:

```
::: {.callout-note title="Your Turn"}
```

```
Something for you to do.
```

```
:::
```

The colours and the icons come from a stylesheet in the book's repo, not from Quarto. Quarto ships five callout types with its own icons, and I have repainted all five.

If you want them in your own work, the docs are at [Quarto:](#)

callout blocks.

! Restart R after a big install

If you install a package that is already loaded in your session, R can end up half on the old version and half on the new one. You then get errors that make no sense and do not survive a restart, which is a miserable way to spend twenty minutes. So once you have worked through the installs below, restart R. In RStudio that is `Cmd / Ctrl + Shift + F10`.

Some boxes are collapsed, showing only their title, like the **Going deeper** one above. Click the title to open one. If you are reading the PDF they are all open already, which is one reason the PDF is longer than it looks online.

The code

Code blocks look like this:

```
library(tidyverse)

airquality |>
  count(Month)
#>   Month    n
#> 1      5 31
#> 2      6 30
#> 3      7 31
#> 4      8 31
#> 5      9 30
```

Lines starting with `#>` are **output**, not code. You do not type those - they are what R prints back, shown so you can check you got the same thing.

Longer blocks have line numbers in the margin so that I can say “look at line 4” and we are both looking at the same line. Short blocks do not, because there is nothing to point at.

Most examples use data that comes with R, like `airquality` above, so you can run anything here without downloading a dataset first.

We use the base pipe, `|>`, rather than `%>%` throughout. If you are used to `%>%`, everything here works the same way.

Where everything lives

- **This book** - <https://r-best-practices.njtierney.com>, and there is a PDF download in the sidebar if you would rather print it or annotate it.
- **The book’s source** - <https://github.com/njtierney/rbestpractices>. Every page has an “Edit this page” link, which goes straight to the

file that made it.

- **The books exercises** - <https://github.com/njtierney/rbp-exercises>. We will discuss how to “automagically” open this in the course

If you find a mistake, please tell me. Every page has a link to open an issue, or you can email me. Errors in teaching material are worth more than errors in almost anything else I write, because thirty people hit them at once. Finding one is a favour, not a complaint.

What to install

If you can, it is worthwhile to install the below before the first session.

We have time on the day, and I would much rather spend ten minutes helping you get set up than have you sit out an exercise. So work through what you can, and bring the rest along.

R

You need **R 4.5.0 or later**, from <https://cran.r-project.org/>. Check what you have with:

```
R.version.string
#> [1] "R version 4.6.1 (2026-06-24)"
```

The reason for 4.5.0 specifically is the penguins. From that version, R ships a **penguins** dataset with base R, so a few of the examples work without installing anything at all. Before 4.5.0, you had to get it from a package.

You also need an editor - the thing you use to edit your R code.

Personally, I recommend [RStudio](#).

If you like, you can use [Positron](#) - for this course I mostly focus on RStudio, but am able to discuss some of the features with positron, which is posit(the company that made RStudio)’s latest generation tool.

If you already have one and you like it, stay where you are.

If you are on an older R

Everything in the course still works, but you will need the [palmer-penguins](#) package for the handful of examples that use **penguins**:

```
install.packages("palmerpenguins")
library(palmerpenguins)
```

One catch: **the column names are different**. When the data moved into base R the names were shortened, so you will need to translate:

In this book	In palmerpenguins
<code>bill_len</code>	<code>bill_length_mm</code>
<code>bill_dep</code>	<code>bill_depth_mm</code>
<code>flipper_len</code>	<code>flipper_length_mm</code>
<code>body_mass</code>	<code>body_mass_g</code>

The units left the names but not the data - `body_mass` is still in grams.

I wrote about this and the rest of what landed in 4.5.0 here: [R version 4.5.0 is out](#).

You can use Ella Kaye's [basepenguins](#) R package to help you move your existing scripts across if you want to use base R penguins.

Separately, some examples read a `penguins.csv` file rather than the dataset. Those use the longer names, because that is what is in the file - not because of your R version.

The R packages

It might will take a few minutes:

```
install.packages(c(
  "tidyverse",
  "here",
  "lintr",
  "styler",
  "spelling",
  "usethis",
  "reprex",
  "goodpractice",
  "sf",
  "pak",
  "sp"
))

install.packages('fnnmate', repos =
  ↪ c('https://milesmbain.r-universe.dev',
  ↪ 'https://cloud.r-project.org'))
```

`sf` and `sp` are only needed for one optional exercises. They are the heaviest things on the list, so if they give you trouble, leave them and let me know.

Code formatters

[Air](#) is an R formatter from Posit. It is not an R package, so it installs from the terminal rather than from R.

See their installation instruction on their website:

But briefly, here are the most popular ways to install:

On macOS and Linux:

```
curl -LsSf https://github.com/posit-dev/air/releases/latest/download/air-installer.sh | sh
```

On Windows:

```
powershell -ExecutionPolicy Bypass -c "irm https://github.com/posit-dev/air/releases/latest/download/air-installer.ps1 | iex"
```

Check it worked:

```
air --version
```

You should get a version number back. If you get “command not found”, the install did not finish, so try it again.

We set up format-on-save together in chapter 2, so there is nothing else to do with air before we start.

If you have trouble installing {air}, we have installed the {styler} R package before, which does the same thing, just not as fast.

Jarl, the linter

Jarl is a fast linter by [Etienne Bacher](#), built on top of Air. Installation instructions are on its site.

If jarl gives you any trouble, do not worry about it. **lintr** is in the package list above and does the same job from inside R, and everything in chapter 2 works with either.

Checking it all worked

Run this in R. It should print TRUE for everything.

```
pkgs <- c(
  "tidyverse", "here", "lintr", "styler",
  "spelling", "usethis", "reprex", "fnnmate",
  "fnnmate", "goodpractice", "sf", "sp"
)

sapply(pkgs, requireNamespace, quietly = TRUE)
```

Then, in the terminal:

```
air --version
```

If both of those look right, you are set.

If something will not install

Please do send me an email before the session rather than fighting with it on your own.

Installation problems are almost always specific to one machine, and they are much faster to solve with two people looking at them. They are also, genuinely, not a reflection on you. Every one of us has lost an afternoon to a package that would not build. Ask me for some horror stories.

1

Project organisation

“Remember, you are always collaborating with your future self.”

This course is all about best practices in R. I want you to know the principles, and have practice at writing clear, maintainable, and reproducible R code.

To build these practices, you need a good base. A good place to work from here is to have a clean workspace. If you have a clean, and tidy and repeatable way of organising your projects, you will have a clearer sense of how to run the code when you come back to it.

It doesn't have to be perfect, but the goal is that by the end of this, you will know how to make your projects just a little better each time.

1.1 Overview

- **Duration** 60 minutes

1.2 Questions

- What is a file path?
- What is a project, anyway?
- Why organise your project?
- What are some common project structures?
- How do you organise your work?
- What are “good enough” common sense principles of project organisation
- Why should every project have a README?
- How to set up RStudio or Positron

Before we get into projects, and organising them, I really think it is worthwhile spending a bit of time talking about **file paths**.

1.3 A common problem: file paths¹

Have you ever run into this problem?

¹ This section draws from “[Workflow](#)” from my course: “[quarto for scientists](#)”.

```
read.csv("my-very-important-data-file-somewhere.csv")
```

```
Warning in file(file, "rt"): cannot open file
'my-very-important-data-file-somewhere.csv': No such file or directory

Error in `file()` :
! cannot open the connection
```

This error is R saying: I do not know where “my-very-important-data-file-somewhere.csv” is.

Practising *good file storage hygiene* will help maintain an order to **files**, **paths**, and **directories**. Having an order there means fewer errors like the above.

Fewer errors means less time fighting strange errors and more time doing the task. It will make you more productive.

But we need to define a few of these terms: files, directories, and paths.

1.3.1 Files

- Something I can open. Examples are: R script, “example.R”, a word doc, “report.docx”, an image “puppy.png”, a README, “README.md”.
- Files have “file extensions, which are the last characters, which tell us what kind of file they are.”.R” is an R script, “.png” is an image file - standing for “portable network graphics”, “.md” is **a**markdown text file.

1.3.2 Directories

- Also known as folders. This comes from the image analogy of putting pieces of paper inside a folder.
- A directory contains files
- A directory can also contain directories

1.3.3 Paths

A path, or “file path”, or “directory path” is the machine-readable directions to where files on your computer live. Kind of like a street address. “Machine readable” just means your computer can read it easily:

- Machine readable: “1811 Macquarie Street, Hobart, Tasmania”
- Not-quite machine readable: “The big white building on the corner with the pine tree in town”

So, the **file path**:

```
/Users/njtierney/Desktop/analysis-2026/data/penguins.csv
```

Describes the location, (the address) of **penguins.csv**.

It might be easier to see this as a tree, or how this file might look in a file explorer on your computer:

1.4 *tree*

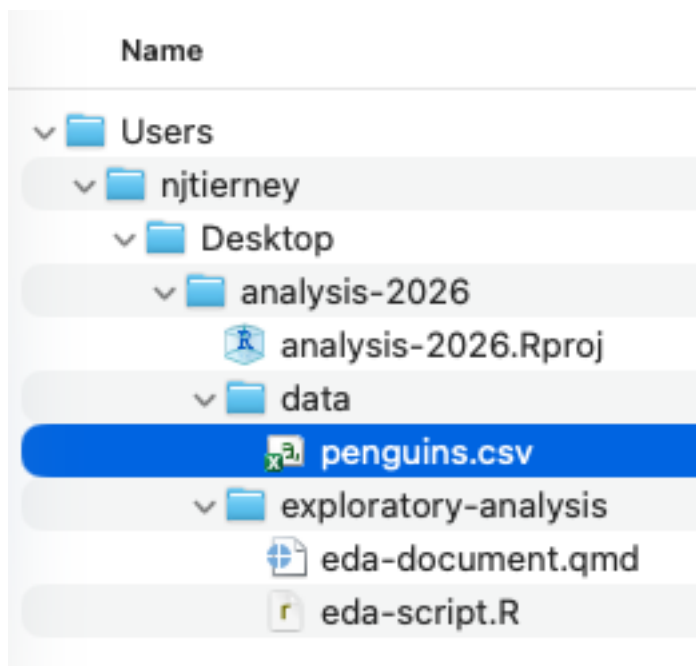
Users

```

`-- njtierney
    |-- Desktop
        |-- analysis-2026
            |-- analysis-2026.Rproj
            |-- data
            |   |-- penguins.csv    << THE FILE IS HERE
            |-- exploratory-analysis
                |-- eda-script.R
                |-- eda-document.qmd

```

1.5 *file explorer*



Example of file and folder structure

Each level is a folder inside the one above it. To read that data file, you might write:

```

penguins <- read_csv("/Users/njtierney/Desktop/analysis-2026/data/penguins.csv")

```

So in the above, the “path” is `"/Users/njtierney/Desktop/analysis-2026/data/penguins.csv"`.

You might also note that sometimes in your work, you don't write down this full path - it is very long! You might do something like:

```
penguins <- read_csv("data/penguins.csv")
```

And this brings us to an important concept - these are two kinds of path:

- **absolute paths** starts from the root of your computer.
- **relative paths** starts from wherever you currently are.

To continue our directions analogy:

- **absolute paths** - the full driving directions to take you from Hobart to Cradle Mountain
- **relative paths** - the directions from where you are (which might be just down the road!) to Cradle Mountain.

So, the code below is using an absolute path:

```
penguins <- read_csv("/Users/njtierney/Desktop/analysis-2_026/data/penguins.csv")
```

The code below is using a relative path:

```
penguins <- read_csv("data/penguins.csv")
```

Of the two, the relative path might look nice, because it is shorter. And that's sort of true.

But the absolute path will work from anywhere on your machine. Every time. Now.

But will it work on your colleagues machine? On your machine in two weeks, next month, next year?

My absolute path will not work on your machine, because your computer (almost certainly) doesn't have a `/Users/njtierney/` folder. It won't work on your own machine after you reorganise your folders when you spring clean. It won't work on the cloud.

i Your Turn

1. Imagine you see the path `/Users/miles/etc1010/week1/data/health.csv`. What are the folders above the file `health.csv`?
2. Run `getwd()`. Where are you?
3. Look at your most recent analysis script. Find every file path in it. How many are absolute?

A common answer to this is that people will often use relative paths, along with a function `setwd()`:

```
setwd("/Users/njtierney/Desktop/analysis-2026")
penguins <- read_csv("data/penguins.csv")
```

Let's talk about why this is **not a good idea**.

1.6 What does `setwd()` do?

Sometimes the first line of a script looks like this:

```
setwd("/Users/njtierney/Desktop/analysis-2026")
```

This says:

Set my working directory to this specific directory, “analysis-2026”, which is in the folders, Users/njtierney/Desktop.

It means you can then read data using the **relative paths** we discussed above:

```
penguins <- read_csv("data/penguins.csv")
```

instead of using the **absolute path**:

```
penguins <- read_csv("/Users/njtierney/Desktop/analysis-2026/
↳ data/penguins.csv")
```

So it does have the effect of **making the file paths work in your file**.

But, it is still a problem, because using `setwd()` like this creates the same problem you had when you used the **absolute path**:

- 0% chance of working on someone else’s machine - **and this could include you, in six months**.
- Means your file is not self-contained or portable. What happens if this folder moves to /Downloads, or onto another machine?

I used to send files to people like this, and you might have come across this yourself - where an R file comes to you and your colleague says:

Yup, all you need to do is **change the `setwd("/Users/njtierney/Desktop/analysis-2026")` to point at where you are, then run it**

So, to get it working, you have to hand-edit the file path for every machine it lands on.

This is painful. And when you do it all the time, it gets old, fast. There is a better way!

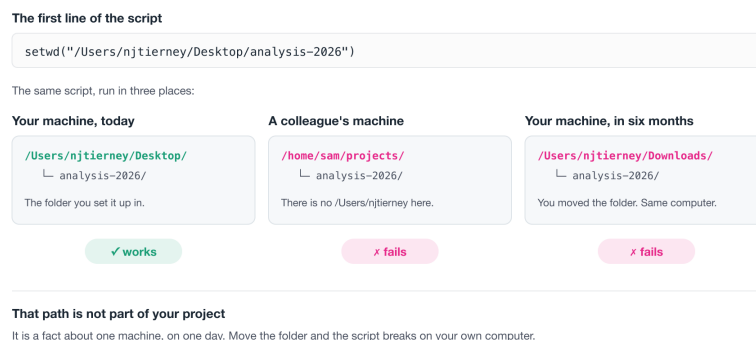


Figure 1.1: The path inside `setwd()` names a folder that only one machine has, and will not work in future places.

This is the heart of the famous line from Jenny Bryan, from her brilliant article, “[Project-oriented workflow](#)”- you should read it:

If the first line of your R script is

```
setwd("C:\\Users\\jenny\\path\\that\\only\\I\\have")
```

I will come into your office and SET YOUR COMPUTER ON FIRE .

1.7 *Is there an answer to the madness?*

Yes.

When you start on a new idea, a new project, a new paper - anything new - it should start its life as a **project**. In RStudio that is an **.Rproj** file; in Positron it is a **workspace folder**.

A project keeps related work together in the same place. They also do the following:

- Keep all your files together.
- Set the working directory to the project directory.
- Start a new session of R.
- Restore previously edited files into your editor tabs.
- Restore other editor settings.
- Let you have several projects open at once.

This helps keep you sane, because:

- Your projects are each independent.
- You can work on different projects at the same time.
- Objects and functions you create in one project won’t affect another.
- You can refer to your data in a consistent way.

And finally, the big one:

Projects resolve file path problems, because they automatically set the working directory to the location of the project.

This means you get two habits from this:

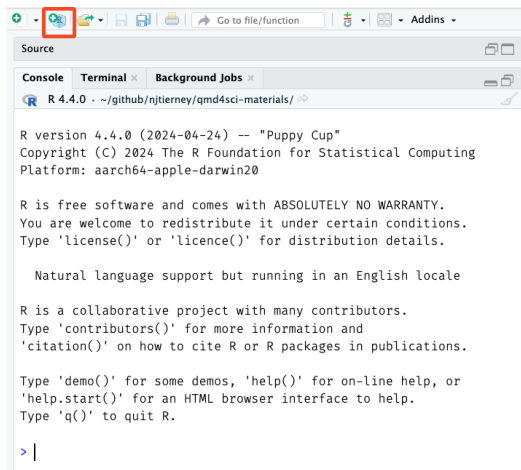
1. **One project, one folder, one .Rproj file.** Open the project by double-clicking the **.Rproj** file, not by opening R and navigating around.
2. **Use relative paths only.** You never call `setwd()` again.

 **Aside: creating a project**

1.8 *RStudio*

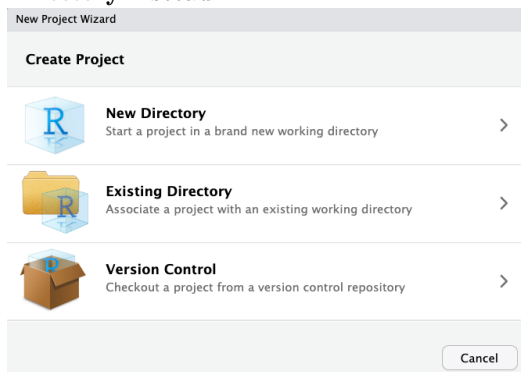
Either use the button in the top right corner, or go:

File → New Project → New Directory → New Project → name your project → Create Project



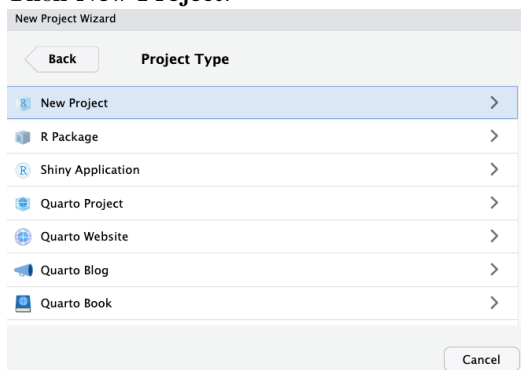
Starting a new project in RStudio.

Then choose **New Directory** if this is a new folder. If you already have a folder you want to turn into a project, choose **Existing Directory** instead.



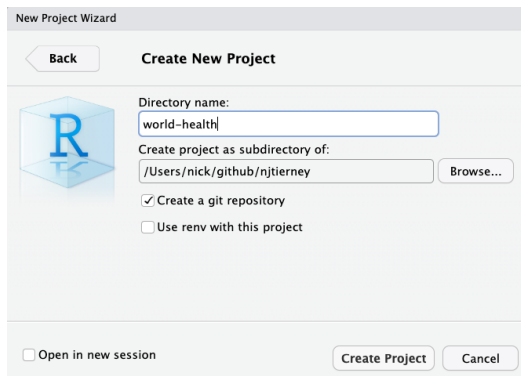
Choosing a new or existing directory.

Then **New Project**.



Choosing the project type.

Then name it, and click **Create Project**.



Naming the project.

1.9 Positron

Positron doesn't use .Rproj files - that is an RStudio specific file. The posit team talk about this in their docs: [“The R Proj File”](#). The gist of this is that Positron does not have Rproj files - you essentially just open a folder and Positron will treat it as a “workspace”.

You can open a folder as a workspace: File -> Open Folder.

If you also want the project to work in RStudio, add an .Rproj file:

```
usethis::use_rstudio()
```

Either way, it is worth spending a couple of minutes on the name. Even a few minutes can make a difference. You want to:

- Keep it short.
- Use no spaces.
- Combine words.

For example, I had a project looking at bat calls, so I called it **screech**, because bats make a screech-y noise. If you are doing global health analysis, maybe it is **world-health**.

1.10 The {here} package

Although projects resolve most file path problems, in some cases you might have many nested folders. To navigate them reliably you can use the [{here}](#) package, which builds the full path from the project root, compactly.

```
here::here("data")
#> [1] "/Users/njtierney/github/njtierney/analysis-2026/d_
↪ ata"
```

and

```
here::here("data", "penguins.csv")
#> [1] "/Users/njtierney/github/njtierney/analysis-2026/d_
↪ ata/penguins.csv"
```

Note that these absolute paths will be different on my computer compared to yours - which is exactly the point.

You can read that [here](#) code as:

In the folder `data`, there is a file called `penguins.csv`. Can you please give me the full path to that file?

This is handy for a few reasons:

1. It makes things *completely* portable.
2. Quarto documents have their own way of looking for files, and this eliminates a whole class of file path pain.
3. If you decide not to use projects, you still have code that works on *any* machine.

So in practice:

```
library(here)
penguins <- read_csv(here("data", "penguins.csv"))
```

works whether the code is called from the top of the project, from inside `analysis/`, or from a Quarto document being rendered in a subfolder - all places where a plain relative path can quietly mean something different.

i Your Turn

1. Open a project of your own, and run `here::here()`. Is it where you expected?
2. Take one absolute path from your most recent script and rewrite it with `here()`.
3. What would have to be true about your working directory for the old version to work?

i Your Turn

When you start a **new** project:

- What is your normal workflow?
- What helps you, what hinders you?
- What do you think you should do?

When you come back to an **old** project:

- What has made it easier to work on it?
- What makes it difficult?
- Anything you wish you did?

There are no wrong answers! I want to have a discussion, and learn what wins, what makes it hard, what makes it OK.

1.11 Why organise your project?

When you are working on a project, you are almost certainly going to be returning to it, at some point. When you do, you might look at the

code, and wonder:

Who wrote this, and are they insane?

And then you realise that indeed, **it was you** that wrote this.

I think it is worthwhile internalising this saying:

You are always collaborating with your future self.²

² I swear I heard ot from someone else,
but I cannot find the source

You want to have notes that keep you organised, and help you return to work smoothly. I think the “Who what when where why how” of story telling can be really useful here:

- **Who** did it: Was it just you? Were there many people?
- **What** you did: what was run
- **When** you did it: Last year, last month?
- **Where** you did it: on your computer? the cloud?
- **Why** you did it: what was the goal
- **How** you did it: What was the order you ran things? Which file should I look at first?

If you can’t answer these questions about a project, how much harder would it be for you to return to it?

It is like making sure your house is tidy, and the dishes are done. It is *technically* optional, sure. But, it does make your life easier when things are tidy, and always in the same spot.

I like to think of this as some self-care for your future self.

What I am getting at here is:

A project that is organised is a project that’s more likely to re-run.

A well organised project does not guarantee reproducibility. But, if you can’t tell me which script to run first, then we cannot reproduce an analysis.

The point is not to tidy your project just so it is tidy. The point is having a tidy project that is well organised makes it easier to check, run later, and understand.

1.12 *Pitfalls of organisation*

There are so many ways to organise a project. And there is no one correct. And organising a project well is hard. While there isn’t a **right** way, there are definitely **better** and **worse** ways. Circling back to the start of the lesson, one of the takeaways from today I want you to remember is:

Your project might not be perfectly organised, but it should be possible for someone to understand how it is structured, and what to run first.

There are a few patterns that I think appear in projects. I am guilty of all of these, so know that I am not judging you if these are how you

work now. What I am saying is that it is OK, and that there are some ways you can improve.

I have put most of these patterns together into one repository, which you can see at:

<https://github.com/njtierney/rbp-materials>

1.12.1 *Everything in one folder*

01-everything-in-one-folder/

```
Rplot.png
Screen Shot 2026-03-14 at 10.23.45 am.png
Untitled1.R
analysis.R
figure1.png
notes.txt
penguins.csv
plot_stuff.R
report draft.docx
results.csv
```

Notice that the R scripts, the data, the figures, a Word document, a screenshot, and `Untitled1.R`, are all *flat* - they are in the same folder. This works until there are more than about fifteen files, at which point finding anything becomes a search problem.

I think about this as like a grocery list: If my shopping list is only 5 items long, it might not impact me when I go to the shops:

- Milk
- Yoghurt
- Muesli
- Bananas
- Muesli bars

But if I'm buying a bunch of food for the week, some meal prep, and a few other bits and pieces, if I don't order the shopping list, and group the similar parts together, I am creating work for myself.

1.13 *Unordered*

- Milk
- Onion powder
- Red capsicum
- Yoghurt
- Muesli
- Bananas
- Muesli bars
- Tomato paste
- Ginger
- Coriander

- Garlic
- Lemongrass
- Coconut milk
- Chicken thighs
- Limes
- Eggplants
- Snow peas
- Thai basil
- 500g beef mince
- Basmati rice

1.14 *Ordered*

- **Produce**
 - Bananas
 - Ginger
 - Garlic
 - Lemongrass
 - Limes
 - Thai basil
 - Coriander
 - Eggplants
 - Red capsicum
 - Snow peas
- **Dairy**
 - Milk
 - Yoghurt
- **Meat**
 - Chicken thighs
 - 500g beef mince
- **Pantry**
 - Muesli
 - Muesli bars
 - Basmati rice
 - Coconut milk
 - Tomato paste
 - Onion powder
- The problem: Flat directory structure can make it hard to group things to understand how they are related.
- The (partial) solution: Put similar things into folders (directories) that give them structure.

1.14.1 *No clear entry point*

```
02-no-clear-entry-point/
  analysis.R
  clean.R
  data.csv
  do_it.R
```

```

final.R
helpers.R
main.R
model.R
plots.R
run.R
script.R
tests.R

```

There are eleven .R files and no way to tell which one runs first. If the only way to know the order they are run in, is only in your head, the project is not reproducible. It is a performance that only you can give.

- The problem: When task order is required, but it is not written down, you cannot reproduce the task.
- The outcome: Encode the task order in another file, such as “make-project.R”, and/or put the files in order.

1.15 *File names encode history, not content*

```

04-names-encode-history/
  analysis.R
  analysis2.R
  analysis_final.R
  analysis_final_USE_THIS_ONE.R
  analysis_final_USE_THIS_ONE_v2 (nick edits).R
  analysis_final_USE_THIS_ONE_v2.R
  analysis_final_v2.R
  penguins.csv

```

The *names* of these files tell a story: the analysis progressed, things changed, and there were concerns about knowing which version worked, and being able to go back and change things. You can think of this as using “track changes” in a word document. This is a kind of version control. But naming the files in order to track versions creates problems, and it is better solved with specific version control software, such as git. I cover this in another course, [“Introduction to git and github”](#).

- The problem: There are multiples of the same file with names that tell you their version, rather than their task.
- The solution: Name the files for what they do, not when you made them. Use a version control system like git

1.16 *Editing raw data in place*

You might have some data provided for analysis and there might be instructions to go through by hand to fix typos, or add new data. You should not make hand edits to the file, as this changes the history and increases the chances of a non-repeatable change. You can make a critical error by accidentally hitting the wrong key. From experience,

having accidentally sorted a single column in an excel sheet and not realising this, I essentially turned a meaningful data source into random noise.

- The problem: Hand editing raw data
- The solution: raw data should be read-only, and changes that happen to it happen with code, and are saved to separate outputs.

1.17 *Outputs mixed in with inputs*

03-outputs-mixed-with-inputs/

```
01-clean.R
02-model.R
cleaned_data.csv
fig-mass-flipper.png
fig-species.png
model_fit.rds
penguins.csv
raw survey data.xlsx
```

In this case, figures, model outputs, and cleaned data sit in the same folder as your raw data and scripts. This means that you cannot tell at a glance what is necessary for your analysis, and what is produced as output by your analysis. If you cannot tell what is input, and what is output, it means you don't have a clear sense of how to reproduce files, or where edits can be safely made. A good test: *could I delete this folder entirely and regenerate it by running my code?* If yes, it is an output, and it should live somewhere that says so.

- The problem: Not knowing what is inputs vs outputs impacts reproducibility.
- The solution: Clearly label inputs and outputs - potentially put your outputs in an “outputs” folder. Or have a file that describes the inputs and outputs, such as a README, or an analysis script.

i Your Turn

Open a project you worked on more than six months ago.

1. Can you tell, within thirty seconds, which script to run first?
2. Which files could you delete and regenerate from code?
3. Which files could you *not* regenerate? Those are the ones that actually matter - are they backed up?

1.18 *Common project structures*

There is no single correct structure, but there is a common shape that most working structures converge on:

The parts worth noticing:

- **data-raw/** holds the raw data, exactly as you received it, along with the script that cleans it. Raw data is read-only, so nothing ever

What kind of thing is this?

The folder names matter less than the kinds of thing they hold. Each row is one kind.

my-project/

my-project.Rproj README.md	start here Open the .Rproj. Read the README.
data-raw/ └─ penguins-raw.csv └─ clean-penguins.R	read only Never edited by hand. Nothing writes here.
data/ └─ penguins.csv	made by code Written by data-raw/clean-penguins.R.
R/ └─ functions.R	used, not run Functions you call from the scripts below.
analysis/ └─ 01-clean-data.R └─ 02-fit-model.R └─ 03-make-figures.R	run in order The numbers are the entry point.
output/ └─ figures/ └─ models/	safe to delete Regenerated by re-running analysis/.

The test: could I delete this folder and get it back by running my code?

Yes for data/ and output/. No for data-raw/ — which is why that is the folder you back up.

writes here, and you never edit it by hand. Keeping the cleaning script next to the raw data means the two never drift apart.

- **data/ holds the analysis-ready data** that the cleaning script produces. This is the version your analysis actually reads.
- **R/ holds functions.** Code that gets *used*, not code that gets *run*. We will spend a whole lesson on this.
- **analysis/ holds scripts that get run**, in order. The numbering is doing real work: it tells your future self the entry point without needing a README.
- **output/ holds things you can delete.** Anything in here can be regenerated by re-running the analysis.

If your project is small, collapse this. Three files in a flat folder with a README is a perfectly good project structure. The point is not to have folders. It is to have *a place where each kind of thing goes*, and to be consistent about it.

💡 The same structure as text, if you want to copy it

```
my-project/
|-- my-project.Rproj
|-- README.md
|-- data-raw/
|   |-- penguins-raw.csv
|   `-- clean-penguins.R
|-- data/
```

Figure 1.2: The same structure, grouped by what each thing *is* rather than what it is called. Raw data is read only, code in R/ gets used, scripts in analysis/ get run in order, and everything in output/ can be deleted and regenerated.

```

|   `-- penguins.csv
|-- R/
|   `-- functions.R
|-- analysis/
|   |-- 01-clean-data.R
|   |-- 02-fit-model.R
|   `-- 03-make-figures.R
`-- output/
    |-- figures/
    `-- models/

```

1.18.1 Naming the files themselves

Naming is hard. We will come back to this throughout the course.

This structure tells you **where** things go, but it does not tell you what to **name** them.

The [files chapter of the Tidyverse style guide](#) is short, and it is worth your time reading. I also really enjoy Jenny Bryan’s talk, “[How to name files like a normie](#)”:

Some especially good pieces of advice:

- **Machine readable.** No spaces, no punctuation beyond - and _, and stick to one case. `01-clean-data.R`, not `01 Clean Data (final).R`.
- **Human readable.** The name should say what the file does. `clean-penguins.R` tells you something. `script2.R` does not.
- **Sorts sensibly.** Numbers at the front, zero padded, so `01`, `02`, `10` stay in order. Dates as `YYYY-MM-DD`, which sorts correctly for free.

That last one is the sneaky one. If your file names sort properly, your file explorer does your project organisation for you, and you get the running order without needing a README.

You still need the README, of course.

i Your Turn

Look at the file names in your most recent project.

1. How many have spaces in them?
2. If you sorted them alphabetically, would the order mean anything?
3. Pick the worst named file and give it a better name.

i Your Turn

Sketch the folder structure for a project you are working on right now. Don’t create it yet - just write it out.

1. Where does raw data go?

2. Where do the things you can regenerate go?
3. Where does someone start reading?

1.19 “Good enough” common sense principles of project organisation

I want to be clear that the goal here is *good enough*, not perfect. This phrasing comes from a paper I recommend to everyone, “[Good enough practices in scientific computing](#)” by Wilson and colleagues. It is aimed at people who have real work to do and no appetite for ceremony.

The principles I would hold you to:

1. **Put each project in its own folder, with its own .Rproj file.**
2. **Treat raw data as read-only.** Never edit it by hand.
3. **Treat generated output as disposable.** If you can’t delete it and get it back, it isn’t really output.
4. **Use relative paths.** Never `setwd()`.
5. **Name files so a stranger can guess what they do.**
6. **Make the running order obvious**, with numbered scripts or a README.
7. **Write a README.**

That is the whole list. If you do those seven things, you are organised enough, and the remaining effort is better spent on the analysis itself.

One more principle, which is really about attitude:

Don’t reorganise everything tonight.

Apply this to your *next* project. Retrofitting a large existing project is a big job with a small payoff, and it is a very effective way to talk yourself out of the whole idea. Start clean, once, and let the habit build.

i Your Turn

Go through the seven principles above and score your most recent project out of seven.

Pick the single one that would have saved you the most time, and write down what you would have to change to fix it.

1.20 Always have a README

A README is a plain text file at the top of your project that answers three questions:

1. **What is this?**
2. **How do I run it?**
3. **Where does the data come from?**

That's it. It does not need to be long. Mine are often ten lines. Here is a complete, entirely adequate README:

```
# Penguin body mass analysis

Fits a model of penguin body mass against flipper length
↪ and species,
for the 2026 report to the department.

## How to run

Open `penguins.Rproj`, then run the scripts in
↪ `analysis/` in order:

1. `01-clean-data.R`
2. `02-fit-model.R`
3. `03-make-figures.R`

Figures are written to `output/figures/`.

## Data

`data/penguins.csv` is from the {palmerpenguins} package,
↪ saved on
2026-03-14. Do not edit this file.
```

The reason I insist on this is that the README is the only part of your project that explains *intent*. Code tells you what happens. The README tells you what you were trying to do, and that is the thing you will have forgotten.

Write it first, if you can. A README written before the analysis is a plan. A README written after the analysis is an archaeology report.

Two refinements worth knowing.

You can have **more than one README**. A short one inside `data-raw/` explaining where those files came from is often the most valuable file in the whole project.

And it's worth recording **when you obtained the data, and what versions of software you used**, so that someone later, including you, can reconstruct similar conditions.

💡 This pays off when the project becomes a paper

Everything in this lesson is framed around your future self. But the same structure is what makes your work usable by *anyone* else, and that matters most at publication.

Karthik Ram and I wrote about this in [Common sense approaches](#)

to sharing tabular data alongside publication (*Patterns*, 2021). The argument, in short: depositing data doesn't make it reusable. What makes it reusable is mundane and structural.

- A README that says who, what, when, where and how.
- **Both** the raw data and the analysis-ready data, each in its own folder.
- The cleaning script that turns one into the other, kept with the raw data.
- Metadata describing each variable: its name, what it means, and its type. Enough to stop someone reading a gene sequence as a date.
- A licence. We recommend CC0.

If you organise your project this way from the start, you don't have to do anything special at submission. You're already most of the way there.

That's the real payoff, and it's why I'd rather you set this up now than tidy it up later.

i Your Turn

Write a README for the project you sketched above. Ten lines. Answer the three questions.

Do it now, before you write any code - you will find it clarifies what you are actually about to do.

1.21 *RStudio and Positron set up*

There are two settings that matter more than all the others, and both are about making sure your R session starts empty every time.

By default, R offers to save your workspace when you quit, and reload it when you start. This sounds helpful. It is not. It means your analysis works because of an object you created three weeks ago and can no longer remember making. When you eventually restart cleanly, the code breaks, and you have no idea why.

1.22 *RStudio*

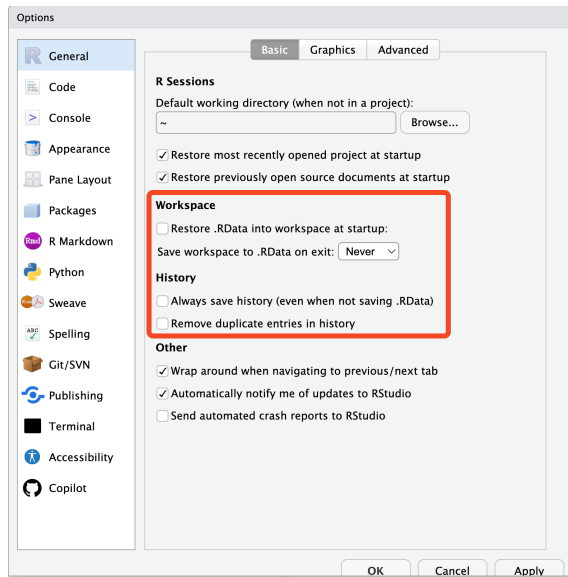
Tools → Global Options (or `Cmd + ,` on macOS) → General.

Under **Workspace**:

- Uncheck **Restore .RData into workspace at startup**
- Set **Save workspace to .RData on exit** to **Never**

Under **History**:

- Uncheck **Always save history (even when not saving .RData)**
- Uncheck **Remove duplicate entries in history**



Setting the options so you neither restore nor save your previous session's work.

1.23 Positron

The same settings live under Settings → Extensions → R. The equivalent behaviour is off by default, so there is usually nothing to change - but it is worth looking, so you know where it is.

This matters for two reasons:

1. **Reproducibility.** You don't want objects from last week cluttering your session, and you don't want an analysis that only works because of one of them.
2. **Privacy.** You don't want private data written into a `.RData` file on disk as a side effect of quitting. You want to read data in, deliberately, every time.

Your history is the list of commands you have typed. Not saving it means you won't come to rely on something you typed in a previous session - which is a good habit to build.

You can also do this from code:

```
usethis::use_blank_slate()
```

With that set, restarting R gives you a genuinely clean session. The keyboard shortcut is worth memorising:

- **Restart R:** Cmd/Ctrl + Shift + F10

The habit that goes with it: **restart R often, and re-run your script from the top.** If it works from a clean session, it works. If it only works in your current session, you have a bug that hasn't happened yet.

This is also the single cheapest check that your project is portable.

Running your scripts in a clean environment is how you find out whether the code quietly depends on something that only exists on your machine. An object you made by hand, a package you loaded and forgot, or your own folder layout.

Nobody else has any of those.

While you are in the settings, two more worth turning on:

- **Show line numbers** - you cannot discuss code you cannot point at.
- **Rainbow parentheses** - cheap insurance against a very common class of error.

i Your Turn

1. Turn off workspace saving using the settings above, or run `usethis::use_blank_slate()`.
2. Restart R with **Cmd/Ctrl + Shift + F10**.
3. Open your most recent analysis script and run it from the top in the clean session.
4. Did it work? If not - and it very often does not - that gap is exactly what the rest of this course is about.

1.24 Summary

- You are always collaborating with your future self.
- A file path is the address of a file; absolute paths only work on one machine.
- Use one folder and one `.Rproj` per project, and relative paths within it.
- `here::here()` builds paths from the project root, wherever the code is called from.
- Raw data is read-only; output is disposable.
- Make the running order obvious.
- Write a README that says what, how, and where from.
- Start R with a blank slate, and restart often.

Next, we will look at what happens *inside* those files: how code is styled, and why consistency makes code readable.

